

# WhereisWaldoAgent 项目学习指南

这是一份**面向学习**的文档。目标不是"告诉你代码在哪一行"，而是带你理解：这个项目用到了哪些技术、每项技术解决什么问题、在代码里是怎么落地的，以及为什么当前效果还不理想、可以从哪些角度改进。

## 0. 一句话概括

给一张超级拥挤的《威利在哪里》(Where's Waldo) 图片，让 AI 自动框出威利的位置。

难点在于：图片极度密集（成百上千个小人），威利又故意被画得不显眼。人眼都要找半天，何况是把整张大图直接丢给 AI——AI 会"看花眼"。

这个项目的核心思路就是模仿人类找威利的方式：

- 1. **先粗看**：扫一眼整张图，判断哪些区域人多、最可能藏威利；
- 2. **再切块**：把可疑区域切成小块，一块一块仔细看；
- 3. **找到疑似目标**：在某个小块里发现"红白条纹"，标记为候选；
- 4. **凑近确认**：把候选区域放大再看一眼，确认"这真的是威利吗"；
- 5. **不确定就缩小范围、切得更细**，重复 2~4，直到确认或放弃。

这套"看不清就放大、不断聚焦"的循环，正是本项目用 **LangGraph** 实现的"智能体（Agent）"。

## 1. 技术全景图

| 技术                 | 作用             | 在本项目的角色                   |
|--------------------|----------------|---------------------------|
| LangGraph          | 编排有状态、可循环的工作流  | 整个 Agent 的"骨架"，把各步骤串成一张图  |
| VLM（视觉语言模型）        | 看图 + 理解 + 输出文字 | 真正负责"看图找威利"的大脑，默认用 Claude |
| Prompt Engineering | 让模型输出结构化结果     | 要求模型只返回 JSON，方便程序解析       |
| Pillow (PIL)       | 图像处理           | 裁剪、缩放、画网格、画框              |
| 并发 + 限流重试          | 加速 + 应对 API 限制 | 同时检测多个小块，遇到限流自动退避         |
| 分治 / 由粗到细搜索        | 算法策略           | 把"大海捞针"拆成"分区精捞"           |

下面逐个展开。

## 2. 核心技术一：LangGraph（智能体骨架）

### 2.1 它解决什么问题？

普通脚本是"从上往下跑一遍就结束"。但找威利需要**循环**：找不到 → 缩小范围 → 再找 → 还找不到 → 继续缩小.....直到找到或放弃。

这种"带状态、会循环、还要根据中间结果决定下一步走哪"的流程，用一堆 `if/else` 和 `while` 写会很乱。

**LangGraph** 就是专门用来优雅地表达这类流程的框架。

它把流程画成一张**有向图**：

- **节点 (Node)**：一个处理步骤（一个函数）
- **边 (Edge)**：步骤之间的流转方向
- **状态 (State)**：一个贯穿全程、被不断更新的"数据包"
- **条件边 (Conditional Edge)**：根据当前状态决定下一步走哪个节点（这就是"会拐弯的智能"）

## 2.2 State：贯穿全程的数据包

文件： `agent/state.py`

LangGraph 的每个节点都接收 State、返回对 State 的"更新"。State 就像一个共享白板，每个节点在上面写下自己的产出，供后面的节点读取。

```
class WaldoState(TypedDict):
    original_image_path: str          # 原图路径
    grid_size: int                    # 当前切割粒度 (3 → 3×3 网格)
    region_grid_sizes: dict           # 每个区域各自的粒度
    focus_regions: list               # 当前重点搜索的区域 [[x,y,w,h], ...]
    region_complexity: list           # 每个区域的"复杂度"评分
    candidates: list                 # 候选结果 (疑似威利的小块)
    verified_result: list | None      # 最终确认的坐标, 没找到就是 None
    iteration: int                   # 当前第几轮
    max_iterations: int              # 最多迭代几轮 (防止死循环)
```

### 学习要点：

- `TypedDict` 只是给字典加上"字段类型提示"，本质还是个普通 dict。
- 节点函数返回的 **dict** 会被合并进 **State**（例如 `return {"candidates": [...]}` 只更新 `candidates` 字段）。
- `max_iterations` 是 Agent 设计的重要安全阀——**任何会循环的 Agent 都必须有迭代上限**，否则可能无限烧钱（每轮都在调用付费 API）。

## 2.3 Graph：把节点连成图

文件： `agent/graph.py`

```
g = StateGraph(WaldoState)

# 注册 7 个节点
g.add_node("analyze", analyze_node)
g.add_node("segment", segment_node)
g.add_node("detect", detect_node)
g.add_node("verify", verify_node)
g.add_node("evaluate", evaluate_node)
g.add_node("calibrate", calibrate_node)
```

```

g.add_node("visualize", visualize_node)

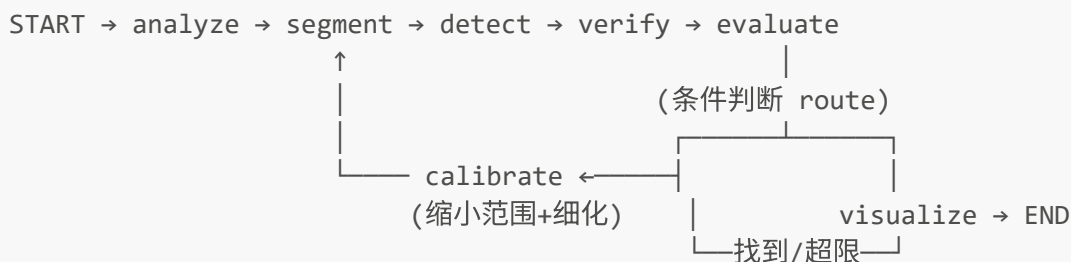
# 普通边：固定流转
g.set_entry_point("analyze")          # 入口
g.add_edge("analyze", "segment")
g.add_edge("segment", "detect")
g.add_edge("detect", "verify")
g.add_edge("verify", "evaluate")
g.add_edge("calibrate", "segment")    # ← 回路！形成循环
g.add_edge("visualize", END)         # 出口

# 条件边：根据 route_after_evaluate 的返回值决定去哪
g.add_conditional_edges(
    "evaluate",
    route_after_evaluate,             # 一个返回 "visualize" 或 "calibrate" 的函数
    {"visualize": "visualize", "calibrate": "calibrate"},
)

graph = g.compile()                   # 编译成可执行图
result = graph.invoke(initial_state)  # 运行

```

画成图就是：



学习要点：

- `add_edge(A, B)` = 固定流转（A 完了一定去 B）。
- `add_conditional_edges` = 智能分叉，这是 Agent 能"自主决策"的关键。
- `calibrate` → `segment` 这条**回边**让图变成了循环，这正是"迭代搜索"的本质。
- 没有条件边的图就只是个"流水线"；有了条件边才叫"Agent"。

## 2.4 条件路由：Agent 的"决策大脑"

文件： `agent/nodes/evaluate.py`

```

def route_after_evaluate(state) -> str:
    if state["verified_result"] is not None:           # 已确认找到
        return "visualize"
    if state["iteration"] >= state["max_iterations"]: # 迭代超限，认输
        return "visualize"
    top = _get_best_candidate(state["candidates"])    # 置信度够高也提前收工
    if top and top.get("verify_confidence", 0) >= 0.85:

```

```
    return "visualize"  
    return "calibrate"
```

# 否则：继续缩小范围再找

这个函数本身不改 State，只读取 State 然后返回一个字符串告诉图"下一步去哪"。这是 LangGraph 里非常典型的写法：用一个纯函数表达"分支逻辑"。

## 3. 核心技术二：VLM（视觉语言模型）

### 3.1 什么是 VLM？

VLM = Vision Language Model，能"看图 + 理解 + 用文字回答"的大模型。你给它一张图 + 一段文字问题，它返回文字答案。本项目用它来回答：

- "这个小块里有没有威利？"（detect 阶段）
- "这张特写图里的人是威利吗？"（verify 阶段）
- "这张图哪些格子人最多？"（analyze 阶段）

### 3.2 图片怎么"发"给模型？——Base64 编码

文件：tools/image\_utils.py

模型 API 走的是 HTTP（传文本），没法直接传二进制图片文件。解决办法：把图片编码成 **Base64** 字符串（一种把二进制变成纯文本的编码方式）。

```
def image_to_base64(image_path, max_size=1568):  
    img = Image.open(image_path).convert("RGB")  
    img = _resize_keep_aspect(img, max_size) # 关键：先缩小！  
    buf = io.BytesIO()  
    img.save(buf, format="JPEG", quality=90)  
    return base64.standard_b64encode(buf.getvalue()).decode("utf-8")
```

**学习要点 max\_size=1568**：Claude / GPT-4o 对图片有尺寸上限，太大的图会被它们自己再压缩、还浪费 token（更贵更慢）。所以发送前先把长边缩到 1568 像素以内。**这也埋下了一个本项目效果不佳的关键伏笔——见第 7 节。**

### 3.3 热插拔架构：统一接口 + 工厂模式

文件：tools/vlm\_client.py

项目不想被某一家模型绑死（万一 Claude 不好用想换 GPT-4o 呢？），所以用了软件工程里经典的\*\*"面向接口编程"\*\*：

```
class BaseVLMClient(ABC):  
    # 抽象基类：定义"所有 VLM 都得会做什么"  
    @abstractmethod  
    def detect(self, image_path) -> DetectResult: ...  
    @abstractmethod  
    def verify(self, image_path) -> VerifyResult: ...
```

```

class ClaudeVLMClient(BaseVLMClient):    # Claude 的具体实现
    def __init__(self, model="claude-sonnet-4-6", ...): ...

class GPT4oVLMClient(BaseVLMClient): ... # GPT-4o 的实现
class GeminiVLMClient(BaseVLMClient): ... # Gemini 的实现

def get_vlm_client(provider="claude"):    # 工厂函数：按名字返回对应实现
    clients = {"claude": ClaudeVLMClient, "gpt4o": GPT4oVLMClient, "gemini":
GeminiVLMClient}
    return clients[provider]()

```

### 学习要点（设计模式）：

- **抽象基类（ABC）** + `@abstractmethod`：强制每个子类都实现 `detect/verify`，否则实例化报错。
- **工厂模式（Factory）**：调用方只说“给我一个 claude 客户端”，不关心内部怎么 new 出来的。
- 好处：业务代码里全是 `vlm.detect(...)`，想换模型只改一个字符串 `provider="gpt4o"`，其余代码一行不动。这就是“热插拔”。

### 3.4 返回结果的封装：dataclass

```

@dataclass
class DetectResult:
    has_waldo: bool           # 有没有威利
    confidence: float         # 置信度 0.0~1.0
    bbox: list[int] | None    # 威利在小块里的坐标 [x,y,w,h]
    raw_response: str = ""    # 模型原始返回 (debug 用)

```

`@dataclass` 自动帮你生成 `__init__` 等样板代码，让“一组相关数据”有清晰的结构，比到处传字典 `{"has_waldo": ..., "confidence": ...}` 安全得多（字段名写错会立刻报错）。

## 4. 核心技术三：Prompt Engineering（让模型输出能被程序读懂）

### 4.1 问题：模型默认爱“啰嗦”

如果你直接问模型“有威利吗？”，它可能回答：

“让我仔细看看.....我注意到右上角似乎有红白条纹，这很可能是威利，不过也可能是.....”

这种自然语言程序没法可靠解析。我们需要模型返回结构化、机器可读的格式——JSON。

### 4.2 解法：在 Prompt 里“约法三章”

文件：tools/vlm\_client.py 的 DETECT\_PROMPT

Does this patch contain Waldo?

Respond ONLY with a JSON object, no markdown, no extra text:

```
{"has_waldo": true/false, "confidence": 0.0-1.0, "bbox": [x, y, width, height] or null}
```

Rules:

- Only set has\_waldo=true if you can clearly see the red-white horizontal stripes
- Be conservative: a false negative is better than a false positive

### 学习要点（提示工程技巧）：

1. **明确格式**：Respond ONLY with a JSON object + 给出 JSON 模板。
2. **给判断特征**：告诉模型威利长什么样（红白横条纹、圆框眼镜、绒球帽），相当于给它“找人标准”。
3. **设定倾向**：a false negative is better than a false positive（宁可漏报也别误报）——这是在用语言调节模型的判断阈值，降低“看到红色就喊威利”的误报。
4. **要置信度**：让模型自评 0~1 的把握程度，程序用它来排序和过滤。

### 4.3 防御性解析：模型不总听话

即便你说了“只返回 JSON”，模型有时还是会包一层 ```json 代码块，或加句废话。所以代码里有容错解析：

```
def _extract_json(text):
    text = re.sub(r"^```(?:json)?\s*", "", text) # 去掉 ```json 开头
    text = re.sub(r"\s*```$", "", text) # 去掉结尾 ```
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        match = re.search(r"\{.*\}", text, re.DOTALL) # 兜底：正则抠出第一个 {...}
        if match:
            return json.loads(match.group())
    return {} # 实在不行返回空，不让程序崩
```

**学习要点**：和 LLM 打交道，永远要假设它的输出可能不规范，做好兜底。analyze 节点甚至有三层 fallback（见 `_parse_grid_scores`：先按 JSON 解析 → 失败则正则抠所有数字 → 再失败则全填 0.5）。

## 5. 核心技术四：图像处理（Pillow）与搜索策略

这是本项目“找威利”的算法核心，分两条线：**怎么切图** 和 **怎么聚焦**。

### 5.1 第一步 analyze：先扫一眼，画“热力图”

文件：`agent/nodes/analyze.py`

人找威利会先扫一眼“哪边人多”。本项目让 VLM 做同样的事：

1. **把原图缩小并画上网格**（`_make_annotated_thumbnail`）：在缩略图上画 2×2 的黄色网格线，并给每格标注 R0C0、R0C1...（行列编号）。

为什么要画网格和标号？这样模型回答时能用我们约定的坐标系说“R0C1 最复杂”，而不是含糊地说“右上角”。

2. 让 VLM 给每格打"复杂度"分 (0~1，越拥挤越高)。
3. 复杂度 → 切割粒度的映射 (`_score_to_grid_size`):

| 复杂度     | grid_size | 含义             |
|---------|-----------|----------------|
| ≥ 0.7   | 3         | 很挤 → 切成 3×3 细看 |
| 0.4~0.7 | 2         | 一般 → 切成 2×2    |
| < 0.4   | 1         | 空旷 → 不切，整块看    |

**学习要点：**这是一种**自适应分配计算资源**的思想——把昂贵的"精细检查"花在最可能有目标的地方， 空旷区域就别浪费 API 调用了。

5.2 第二步 segment：把区域切成小块

文件：`tools/segment.py`、`agent/nodes/segment.py`

把每个 `focus_region` 切成 `grid_size × grid_size` 个小块 (patch)。两个关键细节：

① 重叠切分 (`overlap=0.12`)

```
extend_w = stride_w * overlap    # 每块向右/下多延伸 12%
```

为什么要重叠？因为如果"硬切"，威利可能正好被切在两块的接缝上， 每块都只看到半个威利 → 两边都认不出来。让相邻块重叠 12%，保证威利至少在某一块里是完整的。

② 最小尺寸保护 (`min_patch_size=64`)

```
if stride_w < min_patch_size or stride_h < min_patch_size:
    # 自动减小 grid_size，避免切出比 64×64 还小的碎块
```

防止无限细分——切得太碎，每块就剩几个像素，模型也看不出名堂，纯属浪费调用。

5.3 第三步 detect：逐块检测（核心 API 调用）

文件：`agent/nodes/detect.py`

对每个小块调用 `vlm.detect()`，问"有没有威利"。这一步涉及两个工程技术：

① 并发 (`ThreadPoolExecutor`)

```
with ThreadPoolExecutor(max_workers=MAX_CONCURRENT) as pool:
    future_map = {pool.submit(_call_vlm, vlm, path): ... for ...}
    for future in as_completed(future_map):
        result = future.result()
```

几十个小块如果一个一个串行调用 API，要等很久。线程池可以**同时发起多个请求**。

注意本项目 `MAX_CONCURRENT = 1`（实际还是串行）——因为受限 API 的 50 次/分钟限制，调高并发反而更容易触发限流。**架构支持并发，但当前被限流逼成了串行。**

## ② 限流重试（指数退避 Exponential Backoff）

```
def _call_vlm(vlm, crop_path):
    for attempt in range(MAX_RETRIES):      # 最多重试 4 次
        try:
            return vlm.detect(crop_path)
        except Exception as exc:
            if "429" in str(exc):           # 429 = Too Many Requests (限流)
                wait = RETRY_BASE_WAIT * (2 ** attempt) # 15→30→60→120 秒
                time.sleep(wait)
            else:
                raise
```

**学习要点（指数退避）：**被限流后不要“立刻重试”（只会继续撞墙），而是**每次等待时间翻倍**（15→30→60→120 秒），给服务器喘息时间。这是调用任何外部 API 的标准做法。

检测完后**过滤掉置信度 < 0.15 的块、按置信度从高到低排序**，只留有希望的候选。

## 5.4 第四步 verify：凑近二次确认（降低误报）

文件：agent/nodes/verify.py

这是本项目的**点睛设计——两阶段检测**。detect 阶段模型看的是“小块全图”，容易把别的红白条纹误判成威利。所以对置信度最高的**3 个候选**再做一次精细确认：

1. **坐标换算**：把“威利在小块里的坐标”换算成“在原图里的坐标”（`_patch_to_image_coords`）。
2. **加 padding 扩展**（`_expand_bbox`，向外扩 30%、最小 120px）：只裁威利那一小坨没有上下文，模型不好判断；多留点周围环境，判断更准。
3. **裁出这块特写图，再问一次**：“这确定是威利吗？”（`vlm.verify`）
4. 通过验证的候选里，取置信度最高的作为 `verified_result`。

**学习要点（两阶段 / 粗筛-精排）：**

- detect = **召回**（宁可多抓几个嫌疑，别漏）
- verify = **精确**（在嫌疑里挑出真目标，去掉误报）

这种“先粗后精”的两阶段流水线，在搜索、推荐、目标检测里都极其常见。

## 5.5 第五步 calibrate：找不到就缩小范围、切更细

文件：agent/nodes/calibrate.py

如果 verify 没确认到威利，且还没到迭代上限，就进入“校准”：



```
top_candidates = sorted(candidates, key=lambda c: c["confidence"], reverse=True)
[:3]
new_focus_regions = [c["patch_bbox"] for c in top_candidates] # 只盯最可疑的 3 块
new_grid_size = min(grid_size + 2, 8) # 粒度变细: 3→5→7→8
```

下一轮 segment 就只在这 3 个最可疑的小块里、用更细的网格继续找。这就是**"由粗到细 (coarse-to-fine)"**搜索：每轮都缩小范围、提高分辨率，逐步逼近目标。

## 5.6 第六步 visualize：画框输出

文件：agent/nodes/visualize.py、tools/visualize.py

用 Pillow 在原图上画个红框，保存到 outputs/xxx\_result.jpg。优先用 verified\_result，没有就退而求其次用置信度最高的候选。

## 6. 完整运行流程串讲

把上面串起来，一次完整运行 (python main.py original-images/1.jpg) 大致是：

```
1. initial_state() → 把整张图作为初始搜索区域, iteration=0
2. analyze → VLM 扫 2x2 网格, 给复杂度评分 → 4 个区域 + 各自粒度
3. segment → 按粒度把 4 个区域切成 N 个带重叠的小块
4. detect → 逐块问 VLM"有威利吗", 过滤+排序出候选
5. verify → 对 top-3 候选裁特写、二次确认
6. evaluate + route → 找到了? → visualize 收工
   没找到但还有轮次? → calibrate
7. calibrate → 锁定最可疑的 3 块、粒度+2 → 回到第 3 步 segment
   (重复 3~7, 最多 max_iterations=5 轮)
8. visualize → 画框输出最终结果
```

## 7. ★ 为什么当前效果不理想？（学习重点）

你提到"效果还不理想"。这一节分析**可能的原因和改进方向**——看懂这部分，比看懂代码本身更能提升你对 AI 工程的理解。

### 7.1 致命伤：图片缩放导致威利"糊成一团"

image\_to\_base64 把每张发给模型的图**缩到长边 1568px**。威利在原图里本来就只占几十个像素，原图动辄 4000px 宽——缩到 1568 后，威利可能只剩**十几个像素**，红白条纹彻底糊掉，**模型根本看不清**。

- **改进方向**：detect/verify 阶段发送的应该是**高分辨率的小块截图**，而不是缩小的大图。小块本身尺寸不大，不该被 1568 限制再压一道。可以对小块**放大送入**（超分/上采样），让威利的条纹清晰可见。**这很可能是当前效果差的头号原因**。

### 7.2 初始网格太粗：2x2 扫描漏掉信息

analyze 用 2×2（只有 4 格）评估整张大图的复杂度。对于一张密密麻麻的图，“哪 4 个区域更复杂”这个信号太弱——可能 4 格复杂度都接近，导致后续聚焦没有方向。

- **改进方向：**初始网格调到 3×3 或 4×4，让复杂度热力图更有区分度。

### 7.3 detect 是“分类”而非“定位”，bbox 不可靠

模型看一个小块时，让它同时输出“有没有”+“精确坐标”。但 VLM 并不擅长输出**精确像素坐标**（它没有真正的空间测量能力），返回的 bbox 往往偏移很大，导致 verify 裁错位置、最终框也不准。

- **改进方向：**
  - 把小块切得足够小，让“有威利”约等于“威利就在这块中心”，弱化对精确 bbox 的依赖；
  - 或在 verify 阶段让模型在更小的特写图上重新给一次精确坐标。

### 7.4 阈值是“拍脑袋”定的，没调过

DETECT\_CONFIDENCE\_THRESHOLD=0.15、VERIFY\_CONFIDENCE\_THRESHOLD=0.85 都是经验值。0.15 太低→放进来一堆噪声候选；0.85 太高→真威利也可能被卡掉。

- **改进方向：**用 original-images/ 里已知答案的图做小规模实验，网格搜索这两个阈值，找到准确率最高的组合。这叫**超参数调优**。

### 7.5 限流把并发逼成串行 → 慢

MAX\_CONCURRENT=1 是为了不触发 50 次/分钟限流，但这让检测变得很慢（一张图几十个块、还要多轮迭代，每块一次 API）。慢不直接影响准确率，但影响迭代实验的效率。

- **改进方向：**申请更高的 API 速率档位；或减少每轮块数（更聪明的聚焦）来省调用。

### 7.6 verify 没有把“模型说不是”反馈给搜索

目前 verify 失败后，calibrate 只是单纯“切更细”，但没有利用“这几块被否决了”这个信息去排除它们。

- **改进方向：**把已否决区域记入 State，下一轮主动避开，把算力让给新区域。

### 7.7 缺少评测：没有“准确率”这个数字

最根本的问题：**项目没有一套自动评测**。你说“效果不理想”是凭肉眼感觉，但没有量化指标，就无法判断每次改动是变好还是变差。

- **改进方向（强烈建议第一步做这个）：**
  1. 给 original-images/ 里每张图人工标注威利的真实 bbox（建一个 ground\_truth.json）；
  2. 写个脚本批量跑 Agent，用 IoU（交并比）衡量预测框和真实框的重合度；
  3. 算出“命中率”。有了这个数字，后面所有改进才有依据。

---

## 8. 给学习者的建议路线

如果你想吃透这个项目并把它做好，建议按这个顺序：

1. 先读懂 LangGraph 部分（state.py + graph.py）——理解“图 + 状态 + 条件路由”的范式，这是可迁移到任何 Agent 项目的核心能力。

- 2. 跑通一次：python main.py original-images/1.jpg，看 outputs/ 里各阶段产物 (thumbs/patches/verify)，直观感受每步发生了什么。
- 3. 打开 outputs/patches/ 看几张实际发给模型的小块——你会立刻直观看到"威利糊不糊"（验证第 7.1 节的猜想）。
- 4. 建立评测（第 7.7 节）——这是把"玄学调参"变成"科学优化"的分水岭。
- 5. 逐个验证第 7 节的改进假设，每改一处就用评测脚本量化效果。

9. 关键文件速查

| 想看什么                      | 看哪个文件                    |
|---------------------------|--------------------------|
| Agent 整体结构 / 图怎么连的        | agent/graph.py           |
| 状态数据结构                    | agent/state.py           |
| "先扫一眼定热力图"逻辑              | agent/nodes/analyze.py   |
| 切图算法（重叠、最小尺寸）             | tools/segment.py         |
| 逐块检测 / 并发 / 限流重试          | agent/nodes/detect.py    |
| 两阶段二次确认                   | agent/nodes/verify.py    |
| 决策路由（找到没/继续不继续）           | agent/nodes/evaluate.py  |
| 缩小范围+细化策略                 | agent/nodes/calibrate.py |
| 多模型热插拔 / Prompt / JSON 解析 | tools/vlm_client.py      |
| 图片编码/裁剪/缩放                | tools/image_utils.py     |
| 程序入口                      | main.py                  |

**总结：**这个项目是一个麻雀虽小五脏俱全的 **多模态 Agent** 范例，把 LangGraph 编排、VLM 调用、提示工程、图像处理、并发限流、由粗到细搜索串在了一起。当前效果不佳的瓶颈很可能不在"框架"，而在"喂给模型的图太糊"和"缺少量化评测"这两点——这正是 AI 工程里最常见、也最值得记住的教训：**模型再强，也救不了糟糕的输入和没有度量的盲目优化。**